Deep Learning Steganography: Hiding Trained Models Inside Trained Models

Meghashrita Das
meghashritad@umass.edu

Moumita Karmakar
mkarmakar@umass.edu

Ayesha Binte Mostofa
amostofa@umass.edu

Abstract

We explore an architecture of unified deep network steganography framework for embedding and recovering neural networks within other neural networks. The method operates under both intra-task and inter-task settings using a common pipeline of filter insertion, side information hiding, and partial optimization. We introduce two novel insertion strategies, Random Point Insertion (RPI) and Weight-Based Filter Insertion (WBFI), alongside Gradient-Based Filter Insertion (GFI), to study how filter placement affects performance. Experiments on CIFAR-10, Fashion-MNIST, and Oxford-IIIT Pet show near-lossless recovery in intra-task settings and reasonable recovery in inter-task settings while maintaining cover task performance. All materials related to this project are available at the following GitHub repository.

1 Introduction

As deep neural networks (DNNs) have become valuable computational assets, secure transmission and ownership protection of trained models have emerged as important challenges. Traditional cryptographic methods protect model contents but reveal the existence of confidential communication. Steganography, in contrast, aims to conceal both the message and the presence of communication itself.

Recent advances in deep learning have enabled neural networks to act as steganographic carriers. Existing steganography research has explored tasks such as hiding textual information inside images and using DNN-based methods to distinguish stego-images from clean images. More recently, researchers have explored embedding complete neural networks inside other neural networks using learned encoder–decoder transformations. These methods enable covert model transfer by embedding hidden learning tasks within functional carrier networks while preserving visible model behavior. However, most existing methods focus on hiding a single model and do not thoroughly study how architectural structure affects embedding capacity. In realistic settings, multiple models or learning tasks may need to be transmitted simultaneously. This motivates a new formulation in this project: embedding K secret learning tasks into a single stego model, inspired by secret-sharing principles.

In this work, we propose a deep network steganography framework [3], [4] that supports both intra-task and inter-task settings using a shared pipeline. The pipeline consists of three stages: filter insertion, side information hiding, and partial optimization with statistical regularization. We further investigate different filter placement strategies, including Random Point Insertion (RPI), Weight-Based Filter Insertion (WBFI), and Gradient-Based Filter Insertion (GFI), to understand their effect on performance. We evaluate the framework on standard vision benchmarks, analyzing recovery quality, cover-task accuracy, and robustness across different task settings.

2 Related work

Information hiding has traditionally been studied in classical steganography, where secret messages are embedded within multimedia signals while preserving perceptual similarity. Various types of cover data have been explored in the literature, including images, text, video, audio, and 3D meshes. Most traditional steganographic methods rely on hand-crafted algorithms to subtly modify cover data for data embedding, but they suffer from limited data embedding capacity.

Deep-learning-based steganography improves embedding capacity by jointly learning encoder and decoder networks. Recent DNN-based steganographic methods [1], [2], [5] are able to hide secret images within cover images; however, these approaches typically do not support lossless recovery of the hidden information. Due to the

large size of DNN models, directly hiding them using traditional methods is inefficient and results in significant communication overhead.

Deep learning steganography extends these ideas by enabling a neural network to conceal another neural network through learned parameter transformations. While prior work demonstrates feasibility for single model hiding, it does not address multitask embedding, scalability across the architectures, or robustness under topology changes. In contrast to prior work, we implement deep network steganography framework that supports both intra-task and inter-task settings under a unified pipeline. We further analyze how different filter insertion strategies influence embedding efficiency and recovery performance, providing a more systematic understanding of architectural factors in neural steganography.

3 Technical Implementation

Figure 1 illustrates the overall deep network steganography framework adopted in this work [3]. Alice possesses a trained secret model G_{sec} and embeds it within a stego model G_δ that behaves as a standard neural network for a publicly observable cover task. The stego model is transmitted through a public communication channel where an adversary can inspect the architecture, parameters, and outputs but observes only a benign cover model. Bob, who shares a secret key K with Alice, applies a key-driven extraction procedure to reconstruct the hidden model $\tilde{G}_{\text{sec}}$.

We implement two settings, intra-task and inter-task, within a unified steganographic framework, in which both configurations share a common underlying pipeline comprising filter insertion, side information hiding, and partial optimization, while differing only in the nature of the tasks and their associated learning objectives.

3.1 Steganographic Pipeline

Both intra-task and inter-task settings follow the same three-stage pipeline:

1. Filter Insertion,
2. Side Information Hiding (SIH), and
3. Partial Optimization with Statistical Losses (POS).

The framework transforms a trained secret network into a stego model by inserting additional interference filters

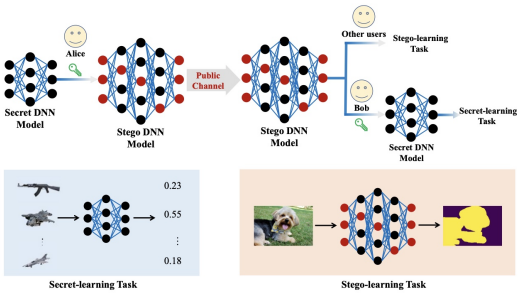

Figure 1: Overview of the deep network steganography framework [3].

while preserving recoverability of the original parameters. The inserted filters are optimized for the cover task, whereas the secret parameters remain protected during stego training.

3.1.1 Stage 1: Filter Insertion

A convolutional layer with d output channels contains $d+1$ valid insertion locations corresponding to positions before, between, or after existing filters. Let $\mathcal{S}$ denote the set of all insertion slots across the network.

For each selected slot, an interference filter is inserted into the corresponding convolutional layer. This operation of inserting filters

1. increases the output dimensionality of the current layer,
2. increases the input dimensionality of the subsequent layer, and
3. initializes the inserted filters using $\mathcal{N}(0, 0.02^2)$ to minimize functional perturbation at initialization.

Gradient-Based Filter Insertion (GFI). GFI ranks insertion locations using gradient information derived from the cover task (Alice possesses a trained secret model G_{sec} with parameter set θ_{sec}. She also possesses a stego training task and dataset D_{st}, and the key K). For each convolutional filter W_i, the average gradient magnitude is computed as

$$g_i = \mathbb{E}_{(x,y)\sim D_{\text{st}}} \left[\left| \nabla_{W_i} \mathcal{L}_{\text{cover}}(G_{\text{sec}}(x), y) \right| \right].$$

The importance score for insertion slot j is defined as

$$\text{score}(j) = \frac{1}{2}(g_{j-1} + g_j),$$

where boundary slots use the nearest valid neighbor. Slots with the highest scores are selected globally across the network. This strategy inserts interference filters near regions that are highly responsive to the cover-task objective, allowing the inserted filters to adapt more effectively during optimization.

Random Point Insertion (RPI). RPI selects insertion locations uniformly at random from $\mathcal{S}$ using a fixed pseudo-random seed. No gradient or structural information is used during selection. This baseline evaluates the contribution of task-aware insertion by removing all informed ranking mechanisms.

Weight-Based Filter Insertion (WBFI). WBFI ranks insertion positions using neighboring filter magnitudes. For each convolutional filter,

$$h_i = \|W_i\|_2,$$

where $\|W_i\|_2$ denotes the ℓ_2 norm of the filter tensor. Slot importance is computed as

$$\text{score}_{\text{WBFI}}(j) = \begin{cases} h_0, & j = 0, \\ \frac{1}{2}(h_{j-1} + h_j), & 1 \le j \le d-1, \\ h_{d-1}, & j = d. \end{cases}$$

The highest-scoring insertion slots are selected globally across all convolutional layers. Unlike GFI, this strategy does not depend on cover-task gradients and instead uses parameter magnitude as a proxy for filter importance.

3.1.2 Stage 2: Side Information Hiding (SIH)

After filter insertion, the stego architecture differs structurally from the original secret model. Consequently, the receiver must recover the locations of original and interference channels during extraction.

For each convolutional layer ℓ, a binary insertion map $m^{(\ell)} \in \{0,1\}^{d_\ell^*}$ is constructed, where 1 denotes an original channel and 0 denotes an interference channel. Concatenating all layer-wise maps forms the global insertion map m. The insertion map is serialized and encrypted using a stream generated from SHA-256(K). A deterministic side-filter location is then derived from the shared secret key using a secondary hash operation. The encrypted payload is embedded into the least significant bits (LSBs) of the selected filter weights. Given a floating-point weight w, SIH modifies the least significant bit after scaling by a factor $s = 10^6$,

$$w' = \frac{\lfloor sw \rfloor_{\text{LSB}}}{s}.$$

The resulting perturbation is bounded by 10^{-6} and produces negligible functional distortion.

3.1.3 Stage 3: Partial Optimization with Statistical Losses (POS)

The POS stage trains the stego model on the cover task while preserving the secret parameters.

A binary mask M is constructed such that parameters belonging to interference filters are assigned value 1, whereas parameters belonging to the original secret model are assigned value 0. During backpropagation, gradients are multiplied element-wise by M, ensuring that only interference parameters are updated.

To regularize the statistical distribution of the stego weights, a clean reference model G_γ with the same widened architecture is trained independently on the cover task. For each conv weight tensor W, denote by $\mu(W)$ and $\sigma(W)$ the per-output-channel mean and standard deviation. Thus, these two statistical regularization terms are

$$L_\mu = \sum_\ell \left\| \mu(W_\delta^{(\ell)}) - \mu(W_\gamma^{(\ell)}) \right\|_2^2, \tag{1}$$

$$L_\sigma = \sum_\ell \left\| \sigma(W_\delta^{(\ell)}) - \sigma(W_\gamma^{(\ell)}) \right\|_2^2. \tag{2}$$

The final optimization objective is defined as

$$\mathcal{L} = \mathcal{L}_{\text{cover}} + \alpha L_\mu + \beta L_\sigma,$$

where $\alpha = 20$ and $\beta = 1$ in all experiments.

3.1.4 Extraction Procedure

To recover the secret model, the receiver first derives the side-filter location using the shared key K. The embedded payload is extracted and decrypted to reconstruct the insertion map. Channels identified as interference filters are subsequently removed layer-by-layer while maintaining

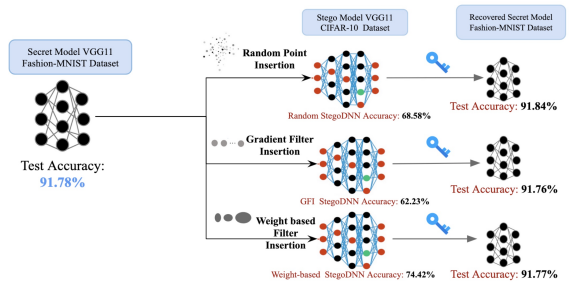

Figure 2: Intra-task deep network steganography framework.

dimensional consistency across neighboring layers. The resulting recovered model $\tilde{G}_{\text{sec}}$ is functionally equivalent to the original secret model except for negligible perturbations introduced during SIH.

3.2 Intra-Task Steganography

The intra-task setting (figure 2) considers secret and cover models performing the same learning objective, namely image classification, but trained on different datasets.

Datasets and Architecture. Fashion-MNIST is used as the secret-task dataset and contains 60,000 training and 10,000 testing grayscale images of size 28×28 distributed across 10 clothing categories. CIFAR-10 serves as the cover-task dataset and contains 50,000 training and 10,000 testing RGB images of size 32×32 across 10 object classes. Both tasks employ a modified VGG11 architecture consisting of eight convolutional layers followed by two fully connected layers.

Insertion Strategy Evaluation. The intra-task experiments evaluate all three insertion strategies:

- Gradient-Based Filter Insertion (GFI),
- Random Point Insertion (RPI), and
- Weight-Based Filter Insertion (WBFI).

All remaining stages of the pipeline, including SIH, POS, and extraction, remain identical across experiments.

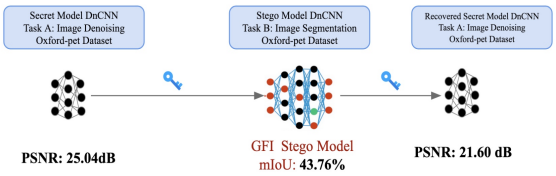

Figure 3: Inter-task deep network steganography framework.

3.3 Inter-Task Steganography

The inter-task setting (figure 3) extends the framework to heterogeneous learning objectives where the secret and cover tasks differ substantially.

Secret and Cover Tasks. The secret task is image denoising using a DnCNN-style residual prediction objective. Given a clean image $x \in [0,1]^{3\times H\times W}$, a noisy observation is generated as

$$\tilde{x} = x + n,$$

where $n \sim \mathcal{N}(0, \sigma^2 I)$ with $\sigma = 50/255$.

The model predicts residual noise $\hat{n}$ and reconstructs the clean image as

$$\hat{x} = \text{clip}(\tilde{x} - \hat{n}, 0, 1).$$

Training minimizes the residual mean-squared error

$$\mathcal{L}_{\text{denoise}} = \mathbb{E}_{x,n}\left[\|\hat{n} - n\|_2^2\right].$$

Performance is evaluated using peak signal-to-noise ratio (PSNR):

$$\text{PSNR}(x, \hat{x}) = 10 \log_{10} \frac{1}{\text{MSE}(x, \hat{x})}.$$

The cover task is semantic segmentation on the Oxford-IIIT Pet dataset using trimap labels corresponding to foreground, border, and background classes. Pixel-wise cross-entropy loss is used during training, while performance is evaluated using mean intersection-over-union (mIoU):

$$\text{mIoU} = \frac{1}{K}\sum_{c=1}^{K} \frac{|P_c \cap T_c|}{|P_c \cup T_c|}.$$

Architecture. The inter-task framework employs a dual-headed DnCNN architecture consisting of a shared

encoder followed by two task-specific output heads.

The encoder contains eight `Conv` $\rightarrow$ `BatchNorm` $\rightarrow$ `ReLU` blocks with 64 channels and 3×3 kernels. Two output heads are attached to the encoder:

- a denoising head for the secret task, and
- a segmentation head for the cover task.

Filter insertion is applied only to encoder layers. The final encoder layer is excluded from insertion to preserve compatibility with the frozen denoising head during extraction.

Pipeline Configuration. Unlike the intra-task setting, the inter-task experiments employ only Gradient-Based Filter Insertion (GFI). Because the secret and cover tasks impose substantially different representational requirements, gradient-driven placement provides insertion locations that better align with the segmentation objective while minimizing disruption to the denoising encoder.

Implementation Details. Two preprocessing pipelines are maintained. Denoising operates directly on raw RGB tensors in $[0, 1]$ space, whereas segmentation inputs are normalized using ImageNet statistics. The architecture exposes separate forward methods for the secret and cover tasks while sharing the same encoder representation.

After filter insertion, the segmentation head is expanded to accommodate the widened encoder dimensionality. Original weights corresponding to the first 64 channels are preserved, while newly introduced channels are initialized to zero. The denoising head remains unchanged because extraction restores the encoder to its original dimensionality prior to secret-task inference.

A key challenge in the inter-task setting arises from Batch Normalization layers within the encoder. Although convolutional weights belonging to the secret network are protected by the partial-update mask, BatchNorm parameters and running statistics remain trainable during stego optimization. Their cumulative drift across multiple layers introduces measurable degradation in recovered denoising performance.

4 Evaluation & Results

The proposed framework is evaluated across three primary dimensions:

1. **Capacity:** the parameter overhead introduced by steganographic embedding, measured using the expansion rate
$$e = \frac{N_\delta}{N_{\text{sec}}},$$
where N_{sec} and N_δ denote the parameter counts of the secret and stego models, respectively.

2. **Cover utility:** the extent to which the stego model preserves performance on the cover task.

3. **Recovery fidelity:** the similarity between the recovered model $\tilde{G}_{\text{sec}}$ and the original secret model G_{sec}, measured using both task performance and relative parameter drift:
$$\text{drift}(\theta_{\text{sec}}, \tilde{\theta}_{\text{sec}}) = \frac{\sum_i |\theta_{\text{sec},i} - \tilde{\theta}_{\text{sec},i}|}{\sum_i |\theta_{\text{sec},i}|}.$$

4.1 Intra-Task Steganography Findings

Metric	Value
Secret VGG11 on Fashion-MNIST	91.57%
Recovered secret on Fashion-MNIST	91.50%
Recovery accuracy gap	-0.07 pp
Relative L_1 weight drift	7.24×10^{-6}
Clean reference G_γ on CIFAR-10	77.01%
Stego G_δ on CIFAR-10	75.79%
Cover utility gap	-1.22 pp
Secret parameters N_{sec}	≈ 9.23M
Stego parameters N_δ	≈ 13.85M
Expansion rate e	1.50

Table 1: Intra-task results at insertion fraction IF $= 0.30$ with $\alpha = 20$, $\beta = 1$, and 12 training epochs per phase.

Recovery Fidelity. The recovered Fashion-MNIST classifier achieves 91.50% accuracy, compared to 91.57% for the original secret model, corresponding to a negligible performance reduction of 0.07 percentage points. The relative parameter drift remains extremely small at 7.24×10^{-6}, indicating that the recovered network is nearly identical to the original model. This residual discrepancy is primarily attributable to the least significant bit (LSB) perturbations introduced during SIH embedding (See table 1).

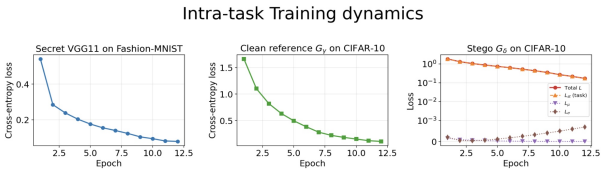

Figure 4: Training dynamics of the intra-task framework.

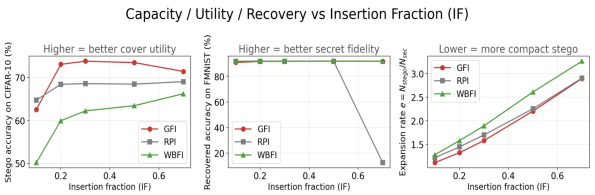

Figure 5: Performance comparison across insertion strategies.

Cover-Task Performance. The stego model achieves 75.79% accuracy on CIFAR-10, compared to 77.01% for the clean reference model trained without steganographic constraints. The resulting cover utility reduction of 1.22 percentage points demonstrates that the stego network preserves strong cover-task functionality despite the restricted optimization imposed by the partial-update mask and statistical regularization terms.

Model Expansion. At an insertion fraction of $\mathrm{IF} = 0.30$, the stego model contains approximately 13.85 million parameters, compared to 9.23 million parameters in the original secret model, yielding an expansion rate of 1.50. Although the embedding process increases model size by approximately 50%, the additional parameter cost remains moderate relative to modern deep neural network architectures.

4.1.1 Training Dynamics

Training curves in figure 4 indicate stable optimization behavior throughout all stages of the intra-task pipeline. The secret classifier's cross-entropy loss decreases monotonically from approximately 0.50 at the beginning of training to 0.08 after 12 epochs on Fashion-MNIST. Similarly, the clean reference model trained on CIFAR-10 converges from 1.65 to 0.30 over the same training duration.

For the stego model, the cover-task loss follows a trajectory similar to that of the clean reference network, while the statistical regularization terms L_μ and L_σ remain consistently small. This behavior indicates that the statistical properties of the stego weights remain close to those of a naturally trained network throughout optimization.

4.1.2 Insertion Strategy Ablation

To evaluate the influence of insertion strategy, we compare Gradient-Based Filter Insertion (GFI) against two alternative baselines across insertion fractions $\mathrm{IF} \in \{0.10, 0.20, 0.30, 0.50, 0.70\}$:

Random Point Insertion (RPI): insertion locations are sampled uniformly at random.
Weight-Based Filter Insertion (WBFI): insertion locations are ranked using neighboring filter ℓ_2 norms.

All experiments employ the same SIH, POS, and extraction pipeline, differing only in the insertion-location selection strategy.

Cover Utility Comparison. Among the evaluated approaches, GFI consistently achieves the strongest cover-task performance. At $\mathrm{IF} = 0.30$, GFI attains approximately 75.8% CIFAR-10 accuracy, compared to 74.4% for WBFI and 68.6% for RPI. These results suggest that gradient-informed placement produces interference filters that are better aligned with the representational structure of the cover task (See figure 5).

Recovery Consistency. All insertion strategies maintain strong recovery performance, with recovered Fashion-MNIST accuracy remaining above 91% whenever SIH re-embedding is performed correctly after stego optimization. Since expansion rate depends only on the number of inserted filters, all methods exhibit identical parameter overhead at a fixed insertion fraction.

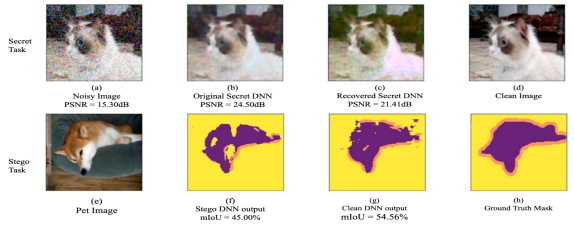

Figure 6: The performance of the secret DNN model and the corresponding stego DNN model on inter-task steganography.

4.2 Inter-Task Steganography Findings

4.2.1 Quantitative Evaluation

Metric	Value
Noise floor (no denoiser)	15.01 dB
Secret DnCNN denoiser PSNR	25.33 dB
Recovered denoiser PSNR	21.70 dB
Recovery loss	−3.63 dB
Clean reference G_γ mIoU	52.77%
Stego G_δ mIoU	43.56%
Cover utility cost	−9.21 pp
Relative L_1 weight drift	3.07×10^{-3}

Table 2: Inter-task results at insertion fraction IF $= 0.30$, $\alpha = 20$, $\beta = 1$, and $\sigma = 50/255$.

Secret-Task Recovery. The secret denoiser achieves 25.33 dB PSNR on the Oxford-IIIT Pet dataset, substantially exceeding the noisy-input baseline of 15.01 dB. After steganographic embedding and extraction, the recovered denoiser achieves 21.70 dB PSNR, corresponding to a degradation of 3.63 dB relative to the original model (See figure 8 and table 2).

Although the recovered network experiences noticeable quality reduction, its performance remains significantly above the noise floor, indicating that the extracted model retains meaningful denoising capability.

Cover-Task Performance. The clean reference segmentation model achieves 52.77% mIoU, whereas the stego model achieves 43.56% mIoU, producing a cover utility reduction of 9.21 percentage points. Compared to the intra-task setting, the inter-task scenario introduces substantially larger degradation, reflecting the representational mismatch between semantic segmentation and image denoising objectives.

Parameter Drift. The inter-task framework exhibits a relative parameter drift of 3.07×10^{-3}, which is substantially larger than the drift observed in the intra-task experiments. Despite using the same SIH and POS mechanisms, the heterogeneous nature of the secret and cover tasks introduces stronger optimization pressure on the shared encoder, leading to increased divergence during stego training.

4.2.2 Sources of Recovery Degradation

The dominant source of degradation arises from Batch Normalization layers within the shared encoder. While the partial-update mask freezes convolutional weights belonging to the secret model, BatchNorm parameters and running statistics remain trainable during stego optimization. Consequently, the BatchNorm layers adapt to the wider feature distribution produced by the stego encoder rather than the original secret encoder configuration.

After extraction, the recovered encoder contains the original convolutional weights paired with BatchNorm statistics optimized for the widened stego architecture. This mismatch propagates across successive normalization layers and significantly impacts denoising quality (See figure 6).

A secondary source of degradation originates from the propagation of SIH-induced perturbations through multiple `Conv` $\rightarrow$ `BatchNorm` $\rightarrow$ `ReLU` blocks. Although the embedded perturbations are individually small, repeated normalization and activation operations amplify their downstream effect.

Finally, the segmentation objective introduces optimization pressure that differs fundamentally from the denoising objective. Semantic segmentation emphasizes object-level abstraction, whereas denoising relies on preserving fine-grained texture information. The resulting representational conflict further contributes to degradation in recovered denoising performance.

4.2.3 Training Dynamics

The secret denoiser converges steadily during training, with residual mean-squared error decreasing from approximately 0.41 at epoch 1 to 0.0033 after 30 epochs (See figure 7). The clean segmentation reference network

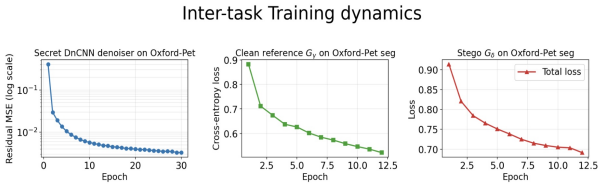

Figure 7: Qualitative results for the inter-task denoising framework.

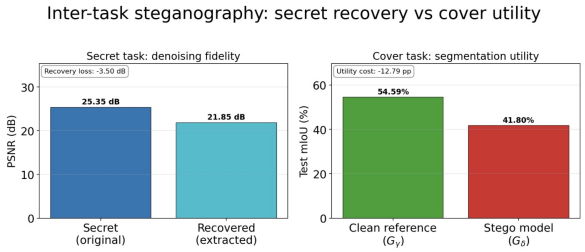

Figure 8: Qualitative segmentation comparison between clean and stego models.

similarly exhibits stable convergence, with cross-entropy loss decreasing from 0.92 to 0.54 over 12 epochs.

For the stego segmentation model, total loss decreases from 0.91 to 0.66 before stabilizing near convergence. The stego loss trajectory closely follows that of the clean reference model but remains consistently higher due to the restricted optimization imposed by the partial-update mask and statistical regularization terms.

Qualitative visualizations further support the quantitative findings. The recovered denoiser produces outputs that are visibly smoother than those generated by the original secret model but remain substantially cleaner than the noisy inputs. Similarly, the stego segmentation outputs preserve overall foreground structure while exhibiting less precise boundary localization compared to the clean reference model.

5 Discussion

Comparison between intra-task and inter-task settings. The intra-task and inter-task experiments use the same steganographic pipeline, but their outcomes differ significantly due to the nature of the tasks and the model architecture. In the intra-task setting, both secret and cover models perform similar classification tasks and the architecture does not include Batch Normalization layers. As a result, the hidden model can be recovered almost perfectly, and the impact on the cover task is minimal.

In contrast, the inter-task setting involves fundamentally different tasks (denoising and segmentation) and uses an encoder with Batch Normalization in every layer. Here, the cover task requires different feature representations than the secret task, and the presence of BatchNorm introduces additional statistical drift during training. Even though the same pipeline is used, the recovery error increases significantly compared to the intra-task case. This shows that the performance difference is mainly due to architectural and task-level mismatch rather than the steganographic pipeline itself.

Key insight. Across both settings, the GFI, SIH, and POS pipeline operates primarily on convolutional weights, since the masking mechanism is designed for these parameters. However, modern neural networks also rely heavily on other components such as BatchNorm parameters, running statistics, and other normalization or scaling layers. These components are not protected by the current masking strategy. As a result, any behavior stored in these unmasked parameters is more likely to change during training, leading to higher recovery drift. This observation suggests a broader principle: the more a network depends on unprotected or dynamically updated components, the harder it becomes to preserve exact recoverability in a steganographic setting.

6 Conclusion

In this work, we analyze a unified steganographic framework for deep neural networks that operates under both intra-task and inter-task settings using a shared pipeline of filter insertion, side information hiding, and partial optimization. Our results show that the method works very effectively when the secret and cover tasks are similar, achieving near-perfect recovery of the hidden model with minimal performance loss. For more challenging inter-task scenarios, the framework still successfully recovers the secret model, although with a larger drop in performance due to task differences and normalization effects. Future work can explore more challenging scenarios, such as embedding multiple secret models into a single stego network, evaluating robustness under compression or fine-tuning attacks, and designing explicit detection methods to distinguish stego models from standard neural networks.

References

[1] Shumeet Baluja. Hiding images within images. *IEEE transactions on pattern analysis and machine intelligence*, 42(7):1685–1697, 2019.

[2] Junpeng Jing, Xin Deng, Mai Xu, Jianyi Wang, and Zhenyu Guan. Hinet: Deep image hiding by invertible network. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4733–4742, 2021.

[3] Guobiao Li, Sheng Li, Meiling Li, Zhenxing Qian, and Xinpeng Zhang. Towards deep network steganography: From networks to networks. *arXiv preprint arXiv:2307.03444*, 2023.

[4] Guobiao Li, Sheng Li, Zhenxing Qian, and Xinpeng Zhang. Cover-separable fixed neural network steganography via deep generative models. *arXiv preprint arXiv:2407.11405*, 2024.

[5] Shao-Ping Lu, Rong Wang, Tao Zhong, and Paul L Rosin. Large-capacity image steganography based on invertible neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10816–10825, 2021.